

EventSpot

Plateforme de recommandation d'événements touristiques

Projet	SAE BUT2 Informatique – Parcours Développement Applicatif
Établissement	IUT de Calais – Université du Littoral
Groupe	Dhesdin Valentin · Gobfert Frédéric · Molinaro Antoine · Regnault Alex · Vanbaelinghem-Dezitter Willem
Dépôt GitHub	https://github.com/Soewyth/sae-web-api
Période	08 Juin 2026 → 12 Juin 2026
Responsables	M. Capitaine · M. Guerrier · M. Delepoulle

Dossier technique de présentation du projet

Table des matières

1.	Thématique choisie	3
2.	Organisation et gestion du projet	4
3.	Modélisation	5
4.	Développement	8
4.1	Moteur de prédiction / recommandation	8
4.2	Automatisation – Seed DataTourisme	12
5.	Explication détaillée du modèle de prédiction	14
6.	Tests	16
7.	Déploiement	18
8.	Chiffrage du projet	20
Annexes		21

1. Thématique choisie

EventSpot – Recommandation d'événements touristiques

Le groupe a choisi la thématique du **tourisme événementiel en France**. L'application **EventSpot** aide un organisateur ou un particulier à choisir la ville et la date idéales pour organiser ou assister à un événement, en combinant données météorologiques historiques, disponibilité des lieux et concurrence événementielle locale.

Justification du choix

- Données réelles disponibles : l'API DataTourisme fournit +300 événements géolocalisés.
- Valeur ajoutée mesurable : le score composite (température, capacité, concurrence) produit une recommandation actionnable.
- Interopérabilité : l'API REST expose les résultats à tout système tiers (front PHP, applications mobiles, etc.).

Sources de données

Source	Données	Accès
DataTourisme API	Événements, villes, géolocalisation (INSEE, lat/lon)	Gratuite · clé API
Open-Meteo Archive	Météo historique quotidienne (temp. min/max, précip., ensoleillement)	Gratuite · sans clé
Wikimedia API	Photo représentative de chaque ville	Gratuite · sans clé

2. Organisation et gestion du projet

Équipe

Nom	GitHub	Responsabilités
Valentin Dhesdin	Soewyth	Schéma Prisma, Seed (réécrit avec distribution aléatoire des reviews), triggers PL/pgSQL (3 + triggers ranking), moteur de recommandation + filtre région, route et controller user, authentification, tests unitaires Jest + PHPUnit (46 tests web), JSON Web Token, cache, CityWeather, architecture API, merge git, documentation README
Gobfert Frédéric	karadocDebian	Frontend – architecture PHP, header/navbar, page Explore, calendrier JS interactif, authentification front, gestion des événements, page Classement (ranking.php), création reviews, persistance données formulaire
Molinaro Antoine	AM2240	Hébergement JrCanDev, MCD/MLD, documentation technique, vidéos, Dockerfile.jrcandev, deploy.sh, création dossier docs
Regnault Alex	RegnaultAlex	Frontend PHP – formulaire homepage, footer, pages about/contact/privacy/terms, style login/register, image de fond, dashboard utilisateur, fix redirections
Vanbaelinghem-Dezitter Willem	Whiplashhh	Makefile, ESLint/Husky/lint-staged, routes Event/Review/Log/Ranking API, city controller, fix env/docker, Swagger, validation IDs, CI/CD webhook, déploiement, log middleware, refactor controllers

Méthodologie

Le projet a été conduit en **présentiel**, sans télétravail, du lundi 8 juin au vendredi 12 juin 2026. Le travail a été organisé avec un dépôt GitHub partagé, en deux branches principales (**backend** et **frontend**) mergées sur **main** en fin de sprint. Les commits sont conventionnels (feat:, fix:, chore:) et chaque membre a publié ses modifications sur le dépôt d'équipe.

Chronologie des contributions (git log)

Synthèse des commits par jour et par membre, extraite du dépôt GitHub.

Jour	Contributeur	Commits / travaux réalisés
Lun. 08/06	Valentin	Init projet, gitignore, structure routers/controllers, auth JWT (register/login)
Lun. 08/06	RegnaultAlex	Modification gitignore frontend
Mar. 09/06	Valentin	Users endpoints, schema Prisma initial, scripts DB Prisma, auth dependencies
Mar. 09/06	Willem (Whiplashhh)	ESLint + Prettier + Husky + commitlint, lint-staged PHP, city controller & router, fix user controller, update env config
Mar. 09/06	Gobfert (karadocDebian)	Init architecture PHP, header, navbar, modifications main.inc
Mar. 09/06	Molinaro (AM2240)	Update Dockerfile.jrcandev
Mar. 09/06	RegnaultAlex	Ajout footer PHP
Mer. 10/06	Valentin	Seed DataTourisme, triggers PL/pgSQL, fetch API météo + calcul score, refactor endpoint unique recommandations, score indoor events
Mer. 10/06	Willem (Whiplashhh)	Upgrade Prisma, fix docker env, recommendation router, event CRUD JWT, fix FK city

Mer. 10/06	Gobfert (karadocDebian)	Page Explore + classe ApiClient
Mer. 10/06	RegnaultAlex	Formulaire homepage, fix valeurs formulaire
Jeu. 11/06	Valentin	maxCapacity schema, tests unitaires (auth/city/event/log/review/user/recommendation), fetch précip. et ensoleillement, merge backend→main
Jeu. 11/06	Willem (Whiplashhh)	fix JWT userId type, Swagger docs, review CRUD JWT, validation IDs, seed Wikipedia images
Jeu. 11/06	Gobfert (karadocDebian)	Calendrier JS interactif et cliquable, authentication front
Jeu. 11/06	Molinaro (AM2240)	deploy.sh, modification Dockerfile
Jeu. 11/06	RegnaultAlex	Contenu home.php, pages about/contact/privacy/terms, style login/register
Ven. 12/06	Valentin	Fix pgdata volume, event count outdoor, fix tri villes, CityWeather seed météo, enrichissement days[] top3 Open-Meteo, filtre région /recommandations, PHPUnit 46 tests web, README, merge final
Ven. 12/06	Willem (Whiplashhh)	Ranking API (cities/events/regions + pagination), log middleware, refactor controllers, endpoint review par événement, endpoint log admin, Swagger mis à jour
Ven. 12/06	Gobfert (karadocDebian)	Lien opérationnel avec API, affichage météo si outdoor, nettoyage fichiers PHP inutiles
Ven. 12/06	RegnaultAlex	Event_types et régions dynamiques, image fond formulaire
Ven. 12/06	Molinaro (AM2240)	Merge jrcandev, amélioration deploy.sh (readiness check + migrations Prisma)

Total : ~100 commits sur 5 jours · branches backend, frontend, classement et jrcandev · merge final sur main le vendredi 12 juin.

Stack technique

Runtime	Node.js 20 / TypeScript 5.9
Framework API	Express 5
ORM	Prisma 7 + adaptateur PrismaPg
Base de données	PostgreSQL 16
Conteneurisation	Docker Compose (dev) · Dockerfile multi-stage (prod)
Frontend	PHP 8 (Apache)
Qualité	ESLint · Prettier · Husky · Jest
Déploiement	JrCanDev · Traefik · webhook GitHub
APIs externes	DataTourisme · Open-Meteo · Wikimedia

3. Modélisation

MCD – Modèle Conceptuel de Données (Merise niveau 2)

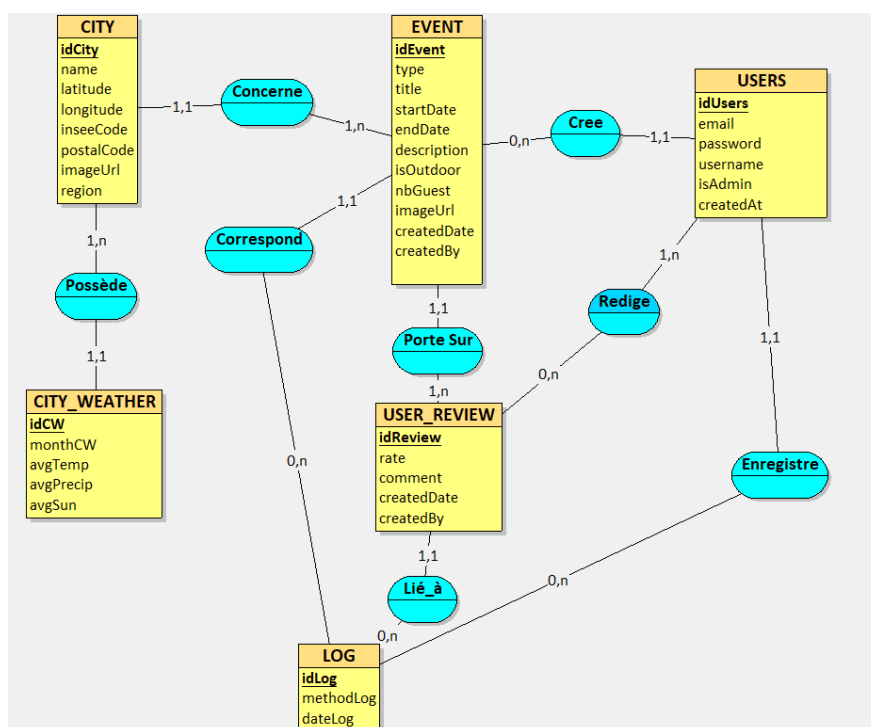
Le MCD décrit les entités métier et leurs associations, indépendamment de toute implémentation technique. Il a été conçu avec le logiciel **Looping**.

Entités

Entité	Attributs
USER	id, email, password, username, isAdmin, createdAt
CITY	id, name, latitude, longitude, inseeCode, postalCode, imageUrl, region
CITY_WEATHER	id, month, avgTemp, avgPrecip, avgSun
EVENT	id, title, type, startDate, endDate, isOutdoor, nbGuests, maxCapacity, imageUrl
USER_REVIEW	id, rating, comment, createdAt
LOG	id, method, date

Associations

Entité source	Cardinalité / Association	Entité cible
CITY	héberge (1,N)	EVENT
CITY	possède (1,N)	CITY_WEATHER
USER	rédige (0,N)	USER_REVIEW
EVENT	reçoit (0,N)	USER_REVIEW
USER	génère (0,N)	LOG
EVENT	référéncé dans (0,N)	LOG
USER_REVIEW	référéncée dans (0,N)	LOG



MLD – Modèle Logique de Données

Le MLD traduit le MCD en tables relationnelles. Les clés primaires sont des **UUID** générés automatiquement. Les clés étrangères appliquent des contraintes **RESTRICT** en suppression et **CASCADE** en mise à jour.

Table	Colonnes & contraintes
User	id UUID PK, email VARCHAR(100) UNIQUE, password VARCHAR(250), username VARCHAR(50), isAdmin BOOL, createdAt TIMESTAMP, createdBy TEXT
City	id UUID PK, name VARCHAR(100), latitude FLOAT, longitude FLOAT, inseeCode VARCHAR(5) UNIQUE, postalCode VARCHAR(5), imageUrl VARCHAR(250), region VARCHAR(100)
CityWeather	id UUID PK, FK_cityId UUID FK→City, month INT, avgTemp FLOAT, avgPrecip FLOAT, avgSun FLOAT — UNIQUE(FK_cityId, month)
Event	id UUID PK, title VARCHAR(100), type EnumType, startDate TIMESTAMP, endDate TIMESTAMP, isOutdoor BOOL, nbGuests INT?, maxCapacity INT?, imageUrl VARCHAR(250)?, FK_cityId UUID FK→City
UserReview	id UUID PK, rating INT (1-5), comment TEXT?, FK_EventId UUID FK→Event, FK_userId UUID FK→User, createdAt TIMESTAMP, createdBy TEXT?
Log	id UUID PK, method EnumMethod, date TIMESTAMP, FK_userId UUID? FK→User, fk_eventId UUID? FK→Event, fk_reviewId UUID? FK→UserReview

Note : La table **CityRanking** (ajoutée en migration 20260612120000) matérialise le classement des villes. Elle est maintenue automatiquement par triggers PL/pgSQL. Clé primaire : FK_cityId (1-1 avec City, suppression en cascade).

Triggers PL/pgSQL

Le projet compte **7 triggers** au total. Les 3 premiers garantissent l'intégrité et la traçabilité. Les 4 suivants maintiennent le classement des villes en temps réel.

Trigger	Déclencheur	Comportement
trg_check_rating	BEFORE INSERT OR UPDATE sur UserReview	Vérifie que rating est entre 1 et 5. Lève EXCEPTION avec valeur reçue sinon.
trg_log_event_insert	AFTER INSERT sur Event	INSERT automatique dans Log (method=POST, fk_eventId=NEW.id, gen_random_uuid()).
trg_log_review_insert	AFTER INSERT sur UserReview	INSERT automatique dans Log (method=POST, fk_reviewId=NEW.id, FK_userId=NEW.FK_userId).
trg_refresh_ranking_event	AFTER INSERT/UPDATE/DELETE sur Event (STATEMENT)	Appelle refresh_city_ranking() — recalcule tout le classement CityRanking après chaque modification d'événement.
trg_refresh_ranking_review	AFTER INSERT/UPDATE/DELETE sur UserReview (STATEMENT)	Appelle refresh_city_ranking() — maintient le classement à jour après chaque modification d'avis.
trg_refresh_ranking_city	AFTER INSERT/DELETE sur City (STATEMENT)	Appelle refresh_city_ranking() — recalcule le classement quand une ville est créée ou supprimée.

Code SQL complet des 3 triggers

```
-- Trigger 1 : Validation du rating (1 à 5) CREATE OR REPLACE FUNCTION check_rating() RETURNS TRIGGER
AS $$ BEGIN IF NEW.rating < 1 OR NEW.rating > 5 THEN RAISE EXCEPTION 'Le rating doit etre compris
entre 1 et 5, valeur recue : %', NEW.rating; END IF; RETURN NEW; END; $$ LANGUAGE plpgsql; CREATE
TRIGGER trg_check_rating BEFORE INSERT OR UPDATE ON "UserReview" FOR EACH ROW EXECUTE FUNCTION
check_rating(); -- Trigger 2 : Log automatique a la creation d'un evenement CREATE OR REPLACE
FUNCTION log_event_insert() RETURNS TRIGGER AS $$ BEGIN INSERT INTO "Log" (id, method, date,
"createdAt", "fk_eventId") VALUES (gen_random_uuid(), 'POST', NOW(), NOW(), NEW.id); RETURN NEW; END;
$$ LANGUAGE plpgsql; CREATE TRIGGER trg_log_event_insert AFTER INSERT ON "Event" FOR EACH ROW EXECUTE
FUNCTION log_event_insert(); -- Trigger 3 : Log automatique a la creation d'un avis CREATE OR REPLACE
FUNCTION log_review_insert() RETURNS TRIGGER AS $$ BEGIN INSERT INTO "Log" (id, method, date,
"createdAt", "fk_reviewId", "FK_userId") VALUES (gen_random_uuid(), 'POST', NOW(), NOW(), NEW.id,
NEW."FK_userId"); RETURN NEW; END; $$ LANGUAGE plpgsql; CREATE TRIGGER trg_log_review_insert AFTER
INSERT ON "UserReview" FOR EACH ROW EXECUTE FUNCTION log_review_insert();
```

4. Développement

4.1 Prédiction – Moteur de recommandation

Le moteur de recommandation est l'élément central de valeur ajoutée du projet. Il expose l'endpoint **GET /recommendations** et retourne le **Top 3 des villes** les plus adaptées à l'organisation d'un événement selon quatre paramètres utilisateur.

Paramètres d'entrée

Paramètre	Type	Rôle
month	Entier 1-12	Mois cible de l'événement
duration	Entier >= 1	Durée en jours
isOutdoor	true / false	Événement en plein air ou en intérieur
nbGuests	Entier >= 1	Nombre de participants attendus

Score Outdoor (isOutdoor = true)

Quand l'événement est en plein air, la météo est prépondérante :

```
scoreGlobal = calcScoreTemp(avgTemp) * 0.5 + calcScoreCapacity(nbGuests, avgMax) * 0.3 +  
calcScoreEvents(nbEvents) * 0.2 Pondération : 50% météo · 30% capacité · 20% concurrence
```

Score Indoor (isOutdoor = false)

En intérieur, la météo n'entre pas en compte :

```
scoreGlobal = calcScoreCapacity(nbGuests, avgMax) * 0.4 + avgEventScore * 0.6 Pondération : 40%  
capacité · 60% concurrence événementielle
```

Fonctions de score

calcScoreTemp(temp) – Courbe trapézoïdale :

```
0°C → 0 (trop froid) 20°C → 100 (idéal) 30°C → 80 (acceptable) 40°C → 0 (canicule) [0→20] : score  
= round(temp / 20 * 100) [20→30] : score = round(100 + (temp-20)/10 * (80-100)) [30→40] : score =  
round(80 + (temp-30)/10 * (0-80))
```

calcScoreCapacity(nbGuests, maxCapacity) :

```
nbGuests > maxCapacity → 0 (lieu trop petit) nbGuests < 100 → 100 (toujours adaptable) sinon : score  
= round((1 - nbGuests/maxCapacity) * 100)
```

calcScoreEvents(count) – Concurrence sur la fenêtre de dates :

```
0-1 événements → 100 (ville libre) 2-3 événements → 50 (concurrence modérée) 4+ événements → 0  
(ville saturée)
```

Table CityWeather – garantie de déterminisme

Pour garantir des résultats **identiques** à chaque appel, les données météo sont **pré-calculées au seed** et stockées dans **CityWeather** (269 villes × 12 mois = 3 228 entrées). Le score est calculé uniquement depuis la BDD, sans appel HTTP. Seul le détail journalier (**days[]**) des 3 villes finalistes effectue 3 requêtes Open-Meteo pour alimenter le calendrier interactif du front.

Cache in-memory (TTL 1h)

Un cache **Map<string, {result, timestamp}>** évite de recalculer pour des paramètres identiques dans la même heure. Clé : month-duration-isOutdoor-nbGuests.

Tri déterministe du Top 3

```
1. Score global DESC 2. avgTemp DESC (tiebreak outdoor) 3. Nom de ville ASC (localeCompare 'fr') 4.
ID ville ASC (UUID - garantit l'unicité absolue)
```

4.2 Automatisation – Seed DataTourisme & CityWeather

Le seed (**api/prisma/seed.ts**) s'exécute via `npm run db:seed` et provisionne entièrement la base de données sans intervention manuelle.

Étapes du seed (ordre d'exécution)

- 1. Purge complète dans l'ordre inverse des FK : Log → UserReview → CityWeather → Event → City → User
- 2. Création des comptes admin (admin@gmail.com) et utilisateur (user@gmail.com) avec hash bcrypt
- 3. Fetch DataTourisme : 3 pages × 100 objets → déduplication des villes par inseeCode (Map<inseeCode, cityData>)
- 4. Enrichissement Wikimedia en parallèle : photo de chaque ville (URL max 250 chars)
- 5. INSERT City séquentiel, stockage de l'ID pour lier les événements
- 6. INSERT Event : 2e passe DataTourisme (label/type/dates), mapping vers EnumType, maxCapacity aléatoire 0-4900
- 7. INSERT UserReview : note 5 (admin) et note 3 (user) par type d'événement présent
- 8. INSERT Log : automatique via triggers PL/pgSQL à chaque INSERT Event/UserReview
- 9. Seed météo CityWeather : batches de 10 villes × 12 mois, délai 500ms entre batches (rate limit Open-Meteo)

Mapping DataTourisme → EnumType

```
Festival → FESTIVAL | Concert → CONCERT Market/SaleEvent → MARCHE | Rambling → RUNNING Conference → CONFERENCE | SportsEvent → SPORT ShowEvent → SPECTACLE | Exhibition → EXPOSITION LocalAnimation → ANIMATION | ScreeningEvent → CINEMA TheaterEvent → THEATRE | (défaut) → ANIMATION
```

Makefile – Commandes disponibles

```
make env # Vérifie que api/.env existe, explique les variables requises make up # Tout-en-un : build + démarrage + migrations + seed (si base vide) make dev # Prépare le poste contributeur (husky + dépendances + prisma generate) make down # Arrête les conteneurs (données conservées) make restart # down + up make logs # Suit les logs API en temps réel (make logs s=db pour un autre service) make migrate # Applique les migrations Prisma (idempotent) make seed # Efface + re-remplit la base make test # Lance les tests Jest dans le conteneur make health # Vérifie GET /api/health make clean # Supprime conteneurs + volumes (DONNÉES PERDUES)
```

Note : `make up` détecte automatiquement si la base est déjà peuplée (COUNT City) et saute le seed si elle ne l'est pas vide, évitant ainsi une réinitialisation accidentelle.

Amélioration du seed (reviews) : La partie reviews a été réécrite. Au lieu de créer systématiquement 2 avis par type d'événement (notes toujours 5 et 3), ~70 % des événements reçoivent désormais 1 à 4 avis avec des notes aléatoires de 1 à 5 et des commentaires contextuels. Résultat : 524 avis bien répartis, scores de villes étalés de 1.4 à 3.7 au lieu de 4/5 ou 0 partout.

4.3 Module Classement – CityRanking

Le module de classement est une fonctionnalité transversale ajoutée en fin de projet, combinant base de données (table matérialisée + triggers), API REST et interface PHP.

Base de données – Table CityRanking

La table **CityRanking** (migration 20260612120000) est une vue matérialisée maintenue automatiquement par 3 triggers PL/pgSQL statement-level. Elle évite tout recalcul à la demande et garantit des lectures $O(1)$ sur le classement.

```
-- Formule de score (migration 20260612150000 - score_v2) score = AVG(notes_evenements) * 0.7 +  
(MIN(nb_evenements, 10) / 10.0) * 5 * 0.3 -- Avantage : une ville avec des evenements mais sans  
review -- n'est plus bloquee a 0 (le bonus evenements rapporte des points)
```

Test de validation : une ville classée 273ème est remontée 1ère après insertion d'un événement noté 5/5, puis rollback — comportement vérifié en base réelle.

API – Module ranking (api/src/ranking/)

Endpoint	Auth	Description
GET /ranking/cities	Non	Classement des villes depuis CityRanking (pagination ?page&?limit, défaut 20)
GET /ranking/events	Non	Tous les événements (300) triés par score, avec nb d'avis (pagination)
GET /ranking/regions	Non	Agrégation par région : score moyen, nb villes, nb événements, photo Wikimedia (cache mémoire)

Chaque endpoint retourne : { result, total, page, totalPages }. Les photos de régions sont récupérées via l'API Wikipédia avec cache mémoire et fallback sur le titre administratif (ex: 'Occitanie (region administrative)').

Frontend PHP – Page Classement (pages/ranking.php)

Page **ranking.php** accessible à tous (contrairement à Explore). 3 onglets en pilules CSS (Villes / Événements / Régions) implémentés comme liens (?tab=cities|events|regions). Chaque tableau affiche : rang, photo, nom, score /5 avec barre de progression (réutilise .score-bar), badges turquoise, nb d'événements/avis. Barre de pagination avec fenêtre à ellipses, disparaît si une seule page.

Filtre par région dans /recommendations

Le paramètre optionnel ?region a été ajouté à l'endpoint **GET /recommendations** pour permettre de filtrer les villes par région administrative. Ce filtre était absent côté backend — corrigé avec validation du type et intégration dans la clé de cache (month-duration-isOutdoor-nbGuests-region).

5. Explication détaillée du modèle de prédiction

Architecture générale

Le modèle est un **système de scoring multicritère à pondérations fixes**. Il ne recourt pas au machine learning mais à une **fonction d'utilité** construite sur des heuristiques métier validées : 20°C est universellement idéal pour un événement extérieur, une salle saturée dissuade le public.

calcScoreTemp – Courbe trapézoïdale

La courbe thermique est de type **trapézoïde**, classique de la logique floue pour modéliser des préférences humaines :

Température (°C)	0	10	20	25	30	35	40
Score	0	50	100	90	80	40	0

La pente [0→20°C] reflète que chaque degré améliore le confort. L'optimum à 20°C est universel. La descente douce [20→30°C] accepte la chaleur modérée. La descente rapide [30→40°C] sanctionne la canicule.

calcScoreCapacity – Ratio de remplissage

Mesure l'**adéquation entre l'affluence prévue et la capacité du lieu**. Un ratio élevé pénalise (peu de marge). Moins de 100 participants = score maximum (très adaptable).

```
ratio = nbGuests / avgMaxCapacity score = (1 - ratio) * 100 Exemple : 500 guests, avgMaxCapacity 2000  
ratio = 0.25 → score = 75 / 100
```

calcScoreEvents – Concurrence événementielle

Évalue si la ville est saturée en événements sur la fenêtre temporelle. Plus il y a de concurrence, moins la ville est attractive (partage du public, des prestataires, de l'attention médiatique).

Événements concurrents	Score	Interprétation
0 ou 1	100	Ville libre
2 ou 3	50	Concurrence modérée
4 et plus	0	Ville saturée

Données météo – Source et fiabilité

Les données proviennent de l'**API Open-Meteo Archive** (réanalyses ERA5, ECMWF). Ce sont des **valeurs mesurées**, non des prévisions. L'hypothèse du modèle : les conditions climatiques d'une année sont représentatives des années futures — acceptable pour une granularité mensuelle.

Champs utilisés par ville/mois : **temperature_2m_max**, **temperature_2m_min**, **precipitation_sum** (mm/jour), **sunshine_duration** (secondes → heures). Moyenne calculée sur tous les jours du mois de l'année N-1.

6. Tests

6.1 Tests API – Jest (TypeScript)

Le projet utilise **Jest** (avec ts-jest) pour les tests unitaires et d'intégration de l'API. **9 suites de tests, 103 tests** au total — dont la nouvelle suite ranking.

```
npx jest tests/ --coverage
```

Résultats de couverture Jest (dernière exécution)

Fichier	% Stmts	% Branch	% Funcs	% Lines	Non couverts
Tous les fichiers	88.2	80.44	91.48	88.84	
src/env.ts	100	100	100	100	
src/auth/auth.router.ts	100	100	100	100	
src/city/city.controller.ts	100	100	100	100	
src/city/city.router.ts	100	100	100	100	
src/common/jwt.middleware.ts	100	100	100	100	
src/common/log.middleware.ts	0	0	0	0	2-76 (non testé)
src/event/event.controller.ts	91.93	91.66	80	91.22	10,152-155,164-167
src/event/event.router.ts	100	100	100	100	
src/log/log.controller.ts	100	100	100	100	
src/ranking/ranking.controller.ts	90.14	64.1	100	92.18	27,36,159,244-245
src/ranking/ranking.router.ts	100	100	100	100	
src/recommendation/recommendation.controller.ts	95.09	90.9	100	97.29	25-26,108-109
src/review/review.controller.ts	100	100	100	100	
src/user/user.controller.ts	63.63	50	66.66	63.63	33-61
src/user/user.router.ts	100	100	100	100	

Note : la couverture globale est 88,2% statements car le nouveau `log.middleware.ts` n'est pas encore couvert par les tests et `user.controller.ts` comporte des fonctions non exercées (33-61). Les seuils globaux (90%) ne sont pas atteints — identifié comme axe d'amélioration.

Résumé des suites de tests Jest

Suite	Statut	Périmètre
tests/auth.test.ts	1 FAIL / reste OK	Un test échoue : champ username inattendu dans la réponse login (mineur, pré-existant)
tests/event.test.ts	PASS	CRUD complet événements
tests/ranking.test.ts	PASS	Endpoints /ranking/cities, /events, /regions — pagination, scores
tests/recommendation.test.ts	PASS	Fonctions de score + endpoint /recommendations + filtre région
tests/user.test.ts	PASS	CRUD utilisateurs
tests/review.test.ts	PASS	CRUD avis + validation rating

tests/city.test.ts	PASS	Listing villes + événements par ville
tests/log.test.ts	PASS	Journal d'activité
tests/none.test.ts	PASS	Bootstrap Jest

Total Jest : 9 suites · 1 failed · 8 passed · 103 tests · 102 passed · Temps d'exécution : 2.25 s

6.2 Tests Frontend – PHPUnit

Une suite de tests PHP a été ajoutée (PHPUnit 11) couvrant la classe **ApiClient** et les helpers de **main.inc.php**. Les appels HTTP réels sont interceptés par un *mock stream wrapper* (`file_get_contents`) — aucune dépendance réseau.

```
make test:web # ou : cd web && vendor/bin/phpunit tests/ --colors=always PHPUnit 11.5.55 by Sebastian
Bergmann Runtime: PHP 8.3.6 Configuration: web/phpunit.xml
..... 46 / 46 (100%) Time: 00:00.012, Memory: 8.00 MB OK (46
tests, 64 assertions)
```

Total PHPUnit : 46 tests · 64 assertions · 100% PASS · 12ms · 0 dépendance réseau.

6.3 Qualité du code

- ESLint + Prettier : lint et formatage automatiques au commit (Husky pre-commit hook)
- TypeScript strict : typage complet des retours Prisma, aucun any non justifié
- Validation HTTP : type et plage des query params vérifiés avant tout traitement
- Log middleware : chaque appel API est journalisé en base (method, route, query, body, user JWT)

7. Déploiement

Architecture de déploiement

L'application est déployée sur **JrCanDev** (infrastructure cloud IUT). Un webhook GitHub déclenche le script **deploy.sh** à chaque push sur la branche **jrcandev**. Le reverse proxy **Traefik** gère le routage HTTPS et les certificats Let's Encrypt automatiquement.

Services Docker Compose (production)

Service	Port	Description
db (postgres:16)	5432 (interne)	PostgreSQL 16, volume nommé persistant postgres_sae_data
api (Node.js 20)	3070 (interne)	API REST, Dockerfile multi-stage (builder+runtime), Traefik label PathPrefix /api
web (PHP/Apache)	80 (interne)	Frontend PHP, volume ./web/src monté, Traefik label /

Procédure de déploiement initial

- 1. Cloner le dépôt : `git clone https://github.com/Soewyth/sae-web-api`
- 2. Récupérer `api/.env` auprès d'un membre de l'équipe (variables listées ci-dessous)
- 3. Lancer tout-en-un : `make up` (build + migrations + seed automatique si base vide)
- 4. Vérifier : `make health` → GET /api/health doit retourner { status: 'ok' }

Variables d'environnement requises (api/.env)

```
DATABASE_URL=postgres://sae_user:sae_password@db:5432/sae_db POSTGRES_USER=sae_user
POSTGRES_PASSWORD=sae_password POSTGRES_DB=sae_db JWT_SECRET=votre_secret_jwt_ici JWT_EXPIRES_IN=1h
DATATOURISME_API_KEY=votre_cle_datatourisme API_PORT=3070 NAME=sae-web-but2-eventspot
```

Déploiement automatique via webhook GitHub

Sur JrCanDev, un webhook pointe vers le serveur. À chaque push sur **jrcandev**, **deploy.sh** est exécuté automatiquement :

```
# deploy.sh (résumé) git fetch origin jrcandev git merge --ff-only origin/jrcandev # ou pull --rebase
si modifications locales docker compose down docker compose -f docker-compose.jrcandev.yml build
--no-cache docker compose -f docker-compose.jrcandev.yml up -d
```

Frontend PHP – Pages disponibles

Fichier	Description
index.php	Page d'entrée principale
pages/home.php	Page d'accueil EventSpot
pages/explore.php	Exploration des événements + recommandations + calendrier
pages/ranking.php	Classement (villes / événements / régions) — 3 onglets + pagination
pages/dashboard.php	Tableau de bord utilisateur connecté
pages/events.php	Liste des événements créés par l'utilisateur

pages/review_create.php	Formulaire de création d'avis (rating 1-5 + commentaire)
pages/reviews.php	Liste des avis
pages/logs.php	Journal d'activité (admin uniquement)
pages/users.php	Gestion des utilisateurs (admin)
pages/update_event.php	Modification d'un événement existant
pages/delete_event.php	Suppression d'un événement
pages/delete_user.php	Suppression d'un utilisateur
pages/book_event.php	Réservation / enregistrement d'un événement
pages/login.php	Connexion utilisateur
pages/register.php	Inscription
pages/logout.php	Déconnexion
pages/about.php	À propos
pages/contact.php	Contact
pages/privacy.php / terms.php	Mentions légales
pages/admin.php	Panneau d'administration
pages/403.php	Page d'erreur accès refusé

8. Chiffrage du projet

Volume de travail

Chaque membre de l'equipe a travaille **35 heures** sur la semaine (7h/jour x 5 jours), soit un volume total de **175 heures** pour les 5 membres.

Nombre de membres	5
Duree	5 jours (lundi 8 -> vendredi 12 juin 2026)
Heures par membre	35 h (7h/jour x 5 jours)
Total heures projet	175 heures

Estimation tarifaire

En prenant comme reference un **TJM junior de 300 EUR/jour** (soit 42,86 EUR/h pour une journee de 7h) :

Taux horaire junior (base 7h/jour, TJM 300 EUR)	~42,86 EUR/h
Cout par membre (35h)	~1 500 EUR
Cout total equipe (5 membres x 35h = 175h)	~7 500 EUR
Licences / APIs	0 EUR (toutes gratuites)
Infrastructure JrCanDev	0 EUR (fournie par l'IUT)
COUT TOTAL DU PROJET	~7 500 EUR

Annexes. Liens et informations d'accès

Liens

Dépôt GitHub	https://github.com/Soewyth/sae-web-api
Site déployé	https://sae-web-but2-eventsport.jrcan.dev/
Documentation API (Swagger)	https://sae-web-but2-eventsport.jrcan.dev/api/api-docs
Vidéo de présentation	https://youtu.be/R3xWG8-pb-Y

Informations d'authentification

Compte admin	Email : admin@gmail.com / Mot de passe : admin
Compte utilisateur	Email : user@gmail.com / Mot de passe : user
Token API	POST /auth/login → champ 'token' dans la réponse JSON (valide 1h)

Routes API complètes

Méthode	Endpoint	Auth	Description
POST	/auth/register	Non	Créer un compte utilisateur
POST	/auth/login	Non	Connexion – retourne un JWT
GET	/user/	JWT	Lister tous les utilisateurs
GET	/user/:id	JWT	Détail d'un utilisateur
GET	/user/:id/events	JWT	Événements créés par un utilisateur
PUT	/user/:id	JWT	Modifier un utilisateur
DELETE	/user/:id	JWT	Supprimer un utilisateur
GET	/city/	Non	Lister toutes les villes
GET	/city/:cityId	Non	Détail d'une ville
GET	/city/:cityId/events	Non	Événements d'une ville
GET	/event/	Non	Lister tous les événements
GET	/event/types	Non	Lister les types d'événements disponibles
GET	/event/:id	Non	Détail d'un événement
POST	/event/	JWT	Créer un événement
PUT	/event/:id	JWT	Modifier un événement
DELETE	/event/:id	JWT	Supprimer un événement
GET	/review/	Non	Lister tous les avis

POST	/review/	JWT	Créer un avis (rating 1-5, validé par trigger)
DELETE	/review/:id	JWT	Supprimer un avis
GET	/logs/	JWT	Journal d'activité (filtres admin)
GET	/recommendations	Non	Top 3 villes recommandées (?region optionnel)
GET	/ranking/cities	Non	Classement des villes (paginé, défaut 20/page)
GET	/ranking/events	Non	Classement des événements — 300 résultats, 15 pages
GET	/ranking/regions	Non	Classement des régions — agrégation + photo Wikimedia
GET	/api/health	Non	Statut API + BDD

Structure du projet

```
sae-web-api/ ■■■■ api/ ■■■■ prisma/ ■■■■ schema.prisma # Modele de donnees (6 tables) ■■■■
seed.ts # Seed complet DataTourisme + meteo ■■■■ migrations/ # Migrations versionnees ■■■■ src/
■■■■ auth/ # JWT login/register ■■■■ city/ # CRUD villes ■■■■ event/ # CRUD evenements ■■■■
■■■■ review/ # CRUD avis ■■■■ log/ # Journal activite ■■■■ recommendation/ # Moteur de
score Top 3 ■■■■ user/ # CRUD utilisateurs ■■■■ common/jwt.middleware.ts ■■■■ client.ts
# PrismaClient singleton ■■■■ index.ts # Point d'entree Express ■■■■ ranking/ # Module
classement (cities/events/regions) ■■■■ common/log.middleware.ts # Journalisation automatique
API ■■■■ tests/ # Jest (9 suites, 103 tests) ■■■■ Dockerfile # Dev ■■■■ Dockerfile.jrcandev #
Production multi-stage ■■■■ web/src/ ■■■■ pages/ # 22 pages PHP (ranking, dashboard, reviews...) ■■■■
■■■■ tests/ # PHPUnit 11 (46 tests, 64 assertions) ■■■■ class/ config/ css/ js/ lib/ ■■■■
index.php / main.inc.php ■■■■ docker-compose.yml # Dev local ■■■■ docker-compose.jrcandev.yml #
Production JrCanDev ■■■■ Makefile # Commandes simplifiees ■■■■ deploy.sh # Script CI/CD webhook
```